BACKEND TECHNICAL TASK

PROGINEER ENGINEERING CHALLENGE

> **Objective:** Build a simple ASP.NET Core Web API that demonstrates fundamental **Authentication, Authorization, JWT, and Middleware** pipelines. The primary focus is evaluating your practical understanding of secure authentication flows, claim verification, role-based authorization, and pipeline route protection.

1 USER REGISTRATION

Create a public registration endpoint that enables users to create an account in the system. Required user parameters include:

- **Full Name:** String field representing the user's name.
- **Email Address:** Unique string identifier used as login username.
- **Password:** User secret credential.

> **CRITICAL PASSWORD SECURITY RULE: Passwords MUST NOT be stored as plain text. Implement secure cryptographic password hashing (e.g., ASP.NET Core PasswordHasher, BCrypt, or PBKDF2).**

2 LOGIN & JWT ISSUANCE

Create a authentication login endpoint accepting `Email` and `Password`. Upon successful validation, the API must generate and return a signed **JSON Web Token (JWT)** containing appropriate identity claims and assigned roles.

3 JWT BEARER AUTHENTICATION

Configure JWT Bearer Authentication to protect endpoints. Implement a user profile endpoint at `GET /api/profile`:

- **Without Token / Invalid Token:** Reject request with `401 Unauthorized` status.
- **With Valid Token:** Decode JWT claims and return profile data with `200 OK` status.

4 ROLE-BASED AUTHORIZATION (RBAC)

Implement a Role-Based Authorization system using at least two roles: **User** and **Admin**. Create role-restricted endpoints:

- `GET /api/user` — Accessible by any authenticated caller holding either **User** or **Admin** role.
- `GET /api/admin` — Accessible exclusively by authenticated users assigned the **Admin** role.

5 MIDDLEWARE & ASP.NET CORE SECURITY PIPELINE

Use ASP.NET Core Middleware and security configuration to define route access rules explicitly across the pipeline:

Method	Endpoint Route	Access Policy Requirement	Expected Behavior
`POST`	`/api/auth/register`	**PUBLIC** (No Auth)	`201 Created` / `400 Bad Request`
`POST`	`/api/auth/login`	**PUBLIC** (No Auth)	`200 OK` / `401 Unauthorized`
`GET`	`/api/profile`	**AUTHENTICATED**	`200 OK` / `401 Unauthorized`
`GET`	`/api/user`	**AUTHENTICATED** (User/Admin)	`200 OK` / `401 Unauthorized`
`GET`	`/api/admin`	**ADMIN ROLE ONLY**	`200 OK` / `403 Forbidden`

6 SWAGGER / OPENAPI SECURITY CONFIGURATION

Configure Swagger/OpenAPI to document authentication requirements and support Bearer Token authorization. Candidates should be able to input a JWT via Swagger's `Authorize` dialog and test protected endpoints directly within the browser UI.

7 POSTMAN INTEGRATION TEST SUITE

Construct a Postman collection covering the full authentication and authorization lifecycle:

Registration & Authentication Cases

- **Register User:** Register a normal user and an admin user.
- **Login Success:** Verify login with correct credentials returns a JWT.
- **Login Failure:** Invalid credentials rejected with `401`.
- **Protected Access:** Access `/api/profile` with valid Bearer token.

Authorization & Role Boundary Cases

- **Unauthenticated Access:** Request protected route without token (returns `401`).
- **User Access:** Normal user accesses `/api/user` (returns `200`).
- **Role Restriction:** Normal user attempts `/api/admin` (returns `403`).
- **Admin Access:** Admin user accesses `/api/admin` (returns `200`).

8 UI & FEATURE EXCLUSIONS (STRICT SCOPE)

> **Backend Only: Do not build any UI or frontend components (no React, HTML, or Razor pages). Keep the project simple and avoid unnecessary complexities like password resets, email verification, refresh tokens, social login, 2FA, or complex permissions systems.**

9 DATABASE & IDENTITY PERSISTENCE

User identities and assigned roles must be persisted in a relational database (e.g., SQL Server, PostgreSQL, SQLite) using Entity Framework Core or direct database scripts. Ensure primary keys, unique email constraints, and clean entity structure.

10 SUBMISSION DELIVERABLES

Your complete submission must include:

1. Complete ASP.NET Core Web API project source code.
2. Database implementation (EF Core migrations or database creation scripts).
3. JWT token generation and validation service implementation.
4. Authentication and authorization middleware pipeline configuration.
5. Swagger/OpenAPI configuration supporting Bearer authentication.
6. Exported Postman collection containing all required positive and negative test cases.
7. Clear `README.md` with instructions on running the app, configuring the database, executing Postman tests, and explaining key design choices.

ProGineer Technologies — WE DELIVER